

BitChill Security Review



Ivan Fitro

June 2nd, 2025

Contents

1 Introduction 3

2 Disclaimer 3

3 About Ivan Fitro 3

4 About BitChill 3

5 Severity classification 3

6 Security Assessment Summary 4

7 Executive Summary 4

8 Findings 6

8.1 Low Findings 6

8.2 Informational Findings 7

1 Introduction

Ivan Fitro conducted a time-limited security assessment of the **BitChill** protocol. This is a second audit focused on reviewing the mitigations from the initial audit and evaluating a new contract that integrates swaps using Uniswap V3.

2 Disclaimer

A smart contract security review cannot ensure the total absence of vulnerabilities. The assessment is inherently limited by time, resources, and available expertise, aiming to identify as many issues as possible within those constraints. While I strive to uncover potential risks, I cannot guarantee 100% security or that any issues will necessarily be found. It is strongly advised to follow up with additional security reviews, implement bug bounty programs, and maintain continuous on-chain monitoring.

3 About Ivan Fitro

Ivan Fitro, also known as **Fitro**, is an independent security researcher specializing in smart contracts. With a track record of uncovering vulnerabilities across various blockchain protocols, he is dedicated to enhancing the ecosystem through diligent security research and thorough audits. You can explore his past work [here](#) or connect with him on Twitter [@FitroIvan](#).

4 About BitChill

BitChill is a protocol built on the Rootstock blockchain, focused on facilitating dollar-cost averaging (DCA) purchases of rBTC. It leverages various stablecoins (ERC-20 tokens), which are deposited into lending protocols to earn interest while waiting to execute scheduled rBTC purchases.

5 Severity classification

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Issues that do not compromise the protocol but aim to improve it are marked as Informational.

Impact - the potential impact of a successful attack includes technical failures, financial losses, and reputational harm

Likelihood - the probability that a given vulnerability is detected and subsequently exploited.

Severity - the overall criticality of the risk

6 Security Assessment Summary

review commit hash - [969afc3698e4120b77e466efc0023266cef6ae5d](#)

Scope

The following smart contracts were in scope of the audit:

- AdminOperations.sol
- DcaManager.sol
- DcaManagerAccessControl.sol
- FeeHandler.sol
- PurchaseMoc.sol
- PurchaseRbtc.sol
- SovrynDocHandler.sol
- SovrynDocHandlerMoc.sol
- SovrynErc20HandlerDex.sol
- TokenHandler.sol
- TokenLending.sol
- TropykusDocHandler.sol
- TropykusDocHandlerMoc.sol
- TropykusErc20HandlerDex.sol
- PurchaseUniswap.sol

7 Executive Summary

Over the course of the security review, Fitro engaged with BitChill to review BitChill. In this period of time a total of **2** issues were uncovered.

Protocol Summary	
Protocol Name	BitChill
Date	June 2nd, 2025

Findings Count	
Severity	Amount
Low	1
Informational	1
Total Findings	2

Audit Findings Summary			
ID	Title	Severity	Status
[L-01]	PurchaseUniswap.sol :: _getAmountOutMinimum() relies on getPrice(), which may return an outdated price, potentially leading to incorrect calculations	Low	Fixed
[I-01]	PurchaseUniswap.sol :: Implement a function to update the oracle address in case the current one stops working	Informational	Fixed

8 Findings

8.1 Low Findings

[L-01] PurchaseUniswap.sol :: _getAmountOutMinimum() relies on getPrice(), which may return an outdated price, potentially leading to incorrect calculations

Description

`_getAmountOutMinimum()` is used to calculate the minimum amount of tokens expected from a Uniswap V3 swap.

```
function _getAmountOutMinimum(uint256 stablecoinAmountToSpend) internal view returns
    (uint256 minimumRbtcAmount) {
    minimumRbtcAmount = (stablecoinAmountToSpend * s_amountOutMinimumPercent) /
        i_MocOracle.getPrice();
}
```

As you can see, the conversion from the stablecoin to rBTC uses `getPrice()` from the MoC Oracle. However, there's no verification to ensure the price is up to date. This can result in using an outdated or incorrect price, leading to an inaccurate calculation of `minimumRbtcAmount`.

This may cause a loss of funds if `minimumRbtcAmount` is lower than it should be, or revert the transaction if it's set too high compared to the correct price.

Recommendations

To resolve the issue, use `getPriceInfo()` instead of `getPrice()`, and validate whether the price is outdated, reverting the transaction if it is.

Note: Implementing this requires adding the `getPriceInfo()` function to `ICoinPairPrice.sol`.

```
function _getAmountOutMinimum(uint256 stablecoinAmountToSpend) internal view returns
    (uint256 minimumRbtcAmount) {
-    minimumRbtcAmount = (stablecoinAmountToSpend * s_amountOutMinimumPercent) /
        i_MocOracle.getPrice();
+    (uint256 currentPrice, isValid, ) = i_MocOracle.getPriceInfo();
+    require(isValid, "Outdated Price");
+
+    minimumRbtcAmount = (stablecoinAmountToSpend * s_amountOutMinimumPercent) /
        currentPrice;
}
```

8.2 Informational Findings

[I-01] PurchaseUniswap.sol :: Implement a function to update the oracle address in case the current one stops working

Description

PurchaseUniswap.sol relies on an oracle to fetch the price of rBTC. However, the oracle address is hardcoded via the constructor and there is no function to update it. If the oracle becomes unavailable or stops functioning, the contract would be unable to retrieve the rBTC price, rendering it inoperable and blocking its core functionality.

Recommendations

Implement an onlyOwner function to allow the oracle address to be updated if needed.